Example reconstruction chain walkthrough

Paul Gessinger
CERN
2025-05-12

Introduction

ACTS Development Setup

- ACTS = A Common Tracking Software
- Two main options for development setup:
  - Native spack (builds packages from source)
  - Docker images (prebuilt dependencies)
- Warning: Spack builds can be challenging due to:
  - ROOT compatibility issues
  - Compiler/OS/library incompatibilities
  - Build recipes not always updated

Docker Setup

Docker Images

- Available prebuilt images with all dependencies
- Choose an image appropriate for your architecture from
  `ghcr.io/acts-project/spack-container`
  - x86_64: `9.2.0_linux-ubuntu24.04-x86_64_gcc-13.3.0`
  - aarch64: `9.2.0_linux-ubuntu24.04-aarch64_gcc-13.3.0` (applies to ARM Macs)
- Pull image using Docker:

```
docker pull ghcr.io/acts-project/spack-container:$IMAGE
```

Setting Up Docker Environment

- Mount working directory into container
- Basic workflow:
  - Start a container with working directory mounted
  - Clone ACTS repository
  - Compile and run ACTS
- Command to start container:

```
docker run --rm -it -v $WORK:/work -w /work $IMAGE
```

- `-v $WORK:/work`: Mounts host directory into container
- `-w /work`: Sets working directory inside container
- `--rm`: Deletes container when stopped
- `-it`: Creates interactive terminal session

Building ACTS

Cloning ACTS

- Clone the repository (inside container for `git-lfs` access)

```
git clone https://github.com/acts-project/acts.git --recursive
```

- `--recursive` flag is important to get OpenDataDetector submodule

Building ACTS

- Use CMake with presets for configuration
- For this tutorial: building Examples framework and OpenDataDetector

```
cmake -S $SOURCE_DIR -B build -G Ninja --preset dev \
  -DACTS_BUILD_UNITTESTS=OFF \
  -DACTS_BUILD_INTEGRATIONTESTS=OFF \
  -DACTS_BUILD_EXAMPLES_PYTHIA8=ON \
  -DACTS_BUILD_EXAMPLES_GEANT4=ON
```

- `$SOURCE_DIR` is your ACTS clone (e.g., `/work/acts`)
- To build the project:

```
cmake --build build
```

Build Optimizations

TIP: Persist `ccache` caches between container runs

- Mount host directory: `-v $WORK:/ccache`
- Do this before first build to benefit

ACTS build requires significant memory

- 2-3GB per CPU core
- For memory issues: Limit cores with `cmake --build -- -j$N`
- Or reconfigure VM resources on macOS

Preparing to Run

Installing Geant4 Data Files

- Required for Geant4 simulation
- Install with:

```
geant4-config --install-datasets
```

TIP: Persist Geant4 data files between container runs

- Mount host directory: `-v $WORK:/g4data`

Setting Up Runtime Environment

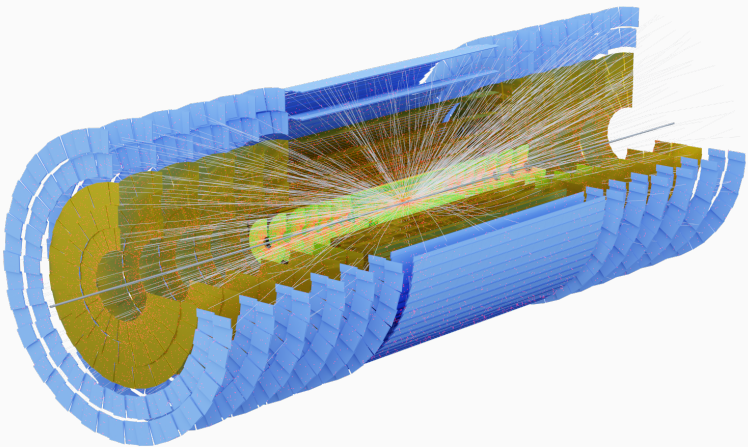
- Load runtime environment before running examples

```
$ source build/this_acts_withdeps.sh
INFO:    Found OpenDataDetector and set it up
INFO:    Acts Python 3.13 bindings setup complete.
```

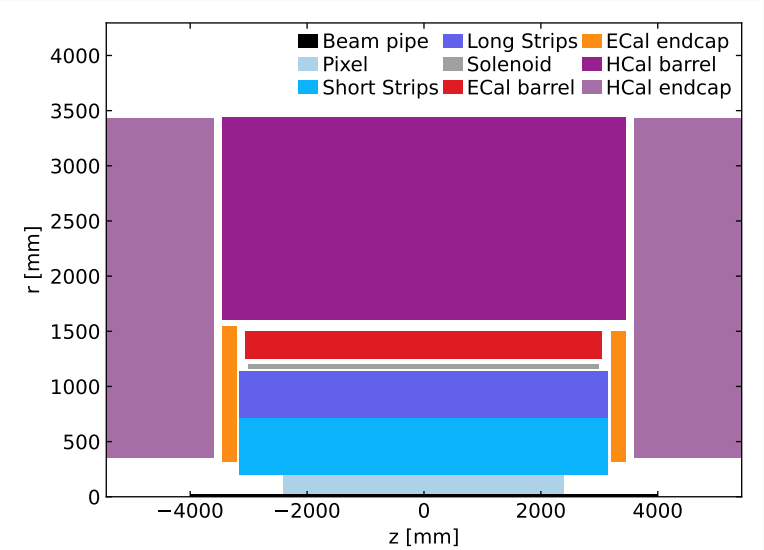
- Sets environment variables for ACTS Examples framework
- Activates dependencies: ROOT, DD4hep, Geant4

Running ODD Example

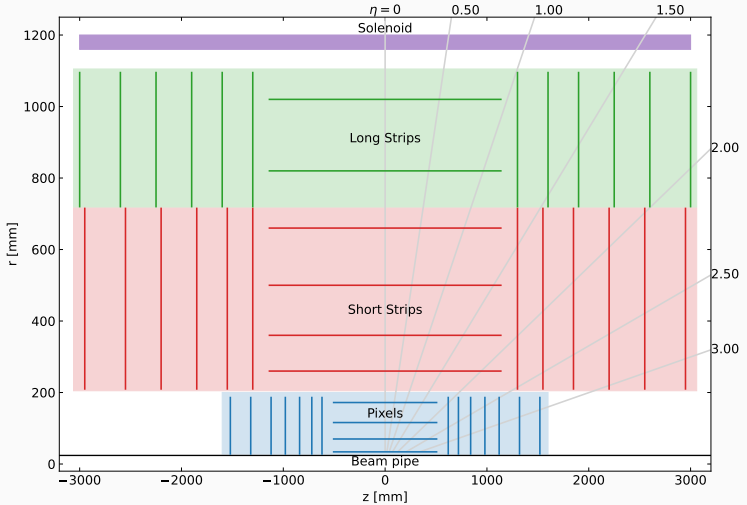
OpenDataDetector



OpenDataDetector



OpenDataDetector



ODD Workflow Example

```
$ Examples/Scripts/Python/full_chain_odd.py --help
usage: full_chain_odd.py [-h] [--output OUTPUT] [--events EVENTS] [--skip SKIP] [--edm4hep EDM4HEP]
                         [--geant4] [--ttbar] [--ttbar-pu TTBAR_PU]
                         [--gun-particles GUN_PARTICLES] [--gun-multiplicity GUN_MULTIPLICITY]
                         [--gun-eta-range GUN_ETA_RANGE GUN_ETA_RANGE]
                         [--gun-pt-range GUN_PT_RANGE GUN_PT_RANGE] [--digi-config DIGI_CONFIG]
                         [--material-config MATERIAL_CONFIG] [--ambi-solver {greedy,scoring,ML}]
                         [--ambi-config AMBI_CONFIG] [--MLSeedFilter] [--reco | --no-reco]
                         [--output-root | --no-output-root] [--output-csv | --no-output-csv]
                         [--output-obj | --no-output-obj]
```

- Default output directory: `$PWD/odd_output`

ODD Workflow Example

Example commands for different simulation scenarios:

- Particle gun with fast simulation:

```
Examples/Scripts/Python/full_chain_odd.py -n10 --gun-multiplicity 10
```

- $t\bar{t}$ events with pile-up (fast simulation):

```
Examples/Scripts/Python/full_chain_odd.py -n10 --ttbar --ttbar-pu 10
```

- $t\bar{t}$ events with pile-up (Geant4 simulation):

```
Examples/Scripts/Python/full_chain_odd.py -n10 --ttbar --ttbar-pu 10 --geant4
```

TIP

- Use `--no-output-csv --no-output-obj` to speed up processing
- This will skip writing the CSV and ROOT files, which can take a while

ODD Script Components

Core Concepts

- Examples framework implements algorithm + event store processing loop
- Main components:
  - **Sequencer**: Runs the event loop, configured with elements
  - **Algorithms**: Main logic building blocks, call ACTS Core algorithms
  - **Readers**: Read data from files or generate events
  - **Writers**: Write data to files in various formats
- Elements exchange information via entries in the event store
- Helper functions in `acts.examples.reconstruction` and `acts.examples.simulation` add sequence elements for specific tasks

Detector Geometry Setup

```
# Figures out the source directory based on the location of this script
geoDir = getOpenDataDetectorDirectory()

# The material map is loaded from the common directory
oddMaterialMap = (
    args.material_config
    if args.material_config
    else geoDir / "data/odd-material-maps.root"
)

# Configure digitization: defines sensitive detector elements and their measurements
oddDigiConfig = (
    args.digi_config
    if args.digi_config
    else geoDir / "config/odd-digi-smearing-config.json"
)
```

Digitization Process

- Configured via JSON file (`oddDigiConfig`)
- Defines which detector elements are sensitive
- Determines what information is measured by each sensor
- Applies smearing to simulate detector resolution
- Example configuration entry:

```
{
  "volume": 16,
  "value": {
    "smearing": [
      { "index": 0, "stddev": 0.015, "type": "Gauss" },
      { "index": 1, "stddev": 0.015, "type": "Gauss" },
      { "index": 5, "stddev": 25, "type": "Gauss" }
    ]
  }
}
```

Detector Geometry Setup (cont.)

```
oddSeedingSel = geoDir / "config/odd-seeding-config.json"
oddMaterialDeco = acts.IMaterialDecorator.fromFile(oddMaterialMap)

# This actually constructs and configures the geometry
detector = getOpenDataDetector(odd_dir=geoDir, mdecorator=oddMaterialDeco)
trackingGeometry = detector.trackingGeometry()
decorators = detector.contextDecorators()

# Configures a constant B field along the z-axis
field = acts.ConstantBField(acts.Vector3(0.0, 0.0, 2.0 * u.T))
# Configure a stable random number sequence
rnd = acts.examples.RandomNumbers(seed=42)

# Configuration of the sequencer
s = acts.examples.Sequencer(
    events=args.events,
    skip=args.skip,
    numThreads=1 if args.geant4 else 1,
    outputDir=str(outputDir),
)
```

Event Generation Options

- Particle Gun (default):
  - Configurable number of particles with specific distributions
  - Properties: p_T, ϕ, η ranges can be set via command line

```
addParticleGun(
    s,
    MomentumConfig(args.gun_pt_range[0] * u.GeV,
                   args.gun_pt_range[1] * u.GeV,
                   transverse=True),
    EtaConfig(args.gun_eta_range[0],
              args.gun_eta_range[1]),
    PhiConfig(0.0, 360.0 * u.degree),
    ParticleConfig(args.gun_particles,
                   acts.PdgParticle.eMuon,
                   randomizeCharge=True),
    vtxGen=acts.examples.GaussianVertexGenerator(
        mean=acts.Vector4(0, 0, 0, 0),
        stddev=acts.Vector4(
            0.0125 * u.mm, 0.0125 * u.mm,
              55.5 * u.mm, 1.0 * u.ns
        ),
    ),
    multiplicity=args.gun_multiplicity,
    rnd=rnd,
)
```

Event Generation Options

- Particle Gun (default):
  - Configurable number of particles with specific distributions
  - Properties: p_T, ϕ, η ranges can be set via command line
- Pythia8 $t\bar{t}$ generation (with `--ttbar`)
  - Generates full $t\bar{t}$ events
  - Configurable pile-up events (with `--ttbar-pu N`)

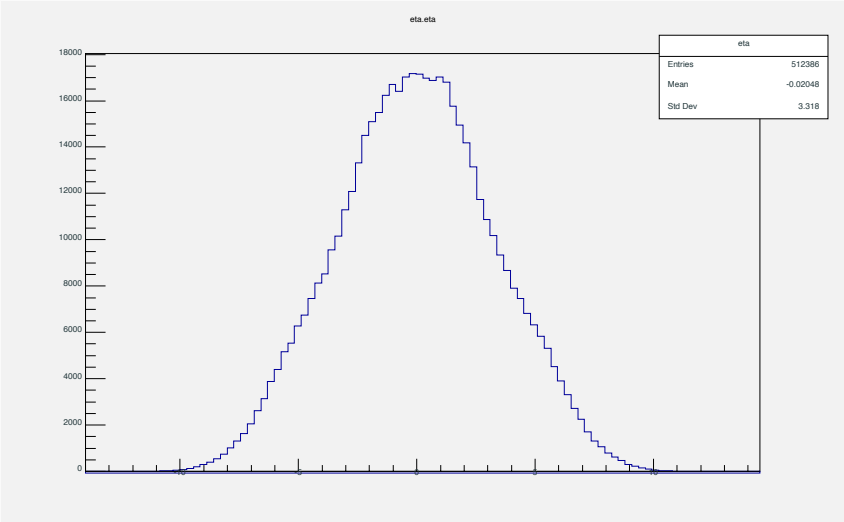
```
addPythia8(
    s,
    hardProcess=["Top:qqbar2ttbar=on"],
    npileup=args.ttbar_pu,
    vtxGen=acts.examples.GaussianVertexGenerator(
        mean=acts.Vector4(0, 0, 0, 0),
        stddev=acts.Vector4(
            0.0125 * u.mm, 0.0125 * u.mm,
            55.5 * u.mm, 5.0 * u.ns
        ),
    ),
    rnd=rnd,
    outputDirRoot=outputDir,
    outputDirCsv=outputDir,
)

addGenParticleSelection(
    s,
    ParticleSelectorConfig(
        rho=(0.0, 24 * u.mm), absZ=(0.0, 1.0 * u.m),
        eta=(-3.0, 3.0), pt=(150 * u.MeV, None),
    ),
)
```

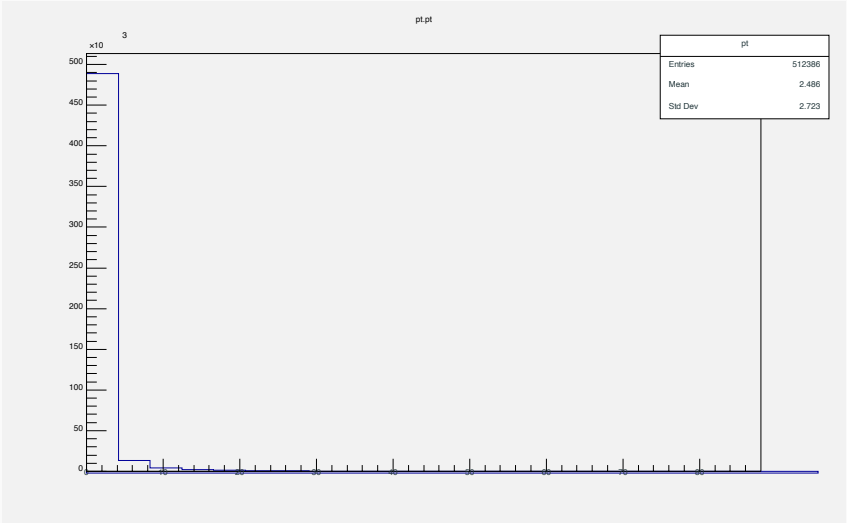
Event Generation Options

- Particle Gun (default):
  - Configurable number of particles with specific distributions
  - Properties: p_T, ϕ, η ranges can be set via command line
- Pythia8 $t\bar{t}$ generation (with `--ttbar`)
  - Generates full $t\bar{t}$ events
  - Configurable pile-up events (with `--ttbar-pu N`)
- EDM4hep input option for existing events (with `--edm4hep`) (**WIP**)

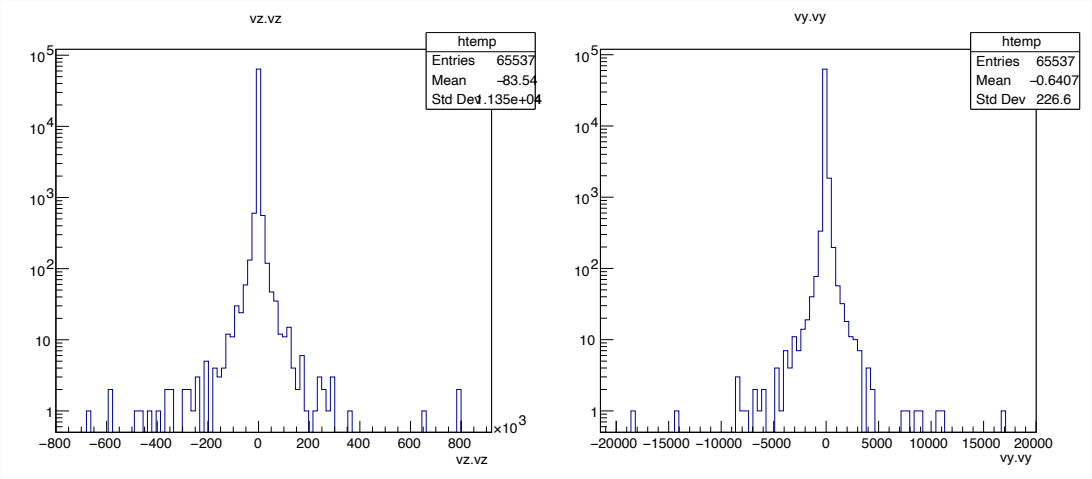
Particle properties



Particle properties



Vertex properties



Simulation Options

- Two simulation backends:
  - **Geant4**: Full detector simulation (with `--geant4`)

```
addGeant4(
      s,
      detector,
      trackingGeometry,
      field,
      outputDirRoot=outputDir,
      outputDirCsv=outputDir,
      outputDirObj=outputDir,
      rnd=rnd,
      killVolume=\
        trackingGeometry.highestTrackingVolume,
      killAfterTime=25 * u.ns,
  )
```

Simulation Options

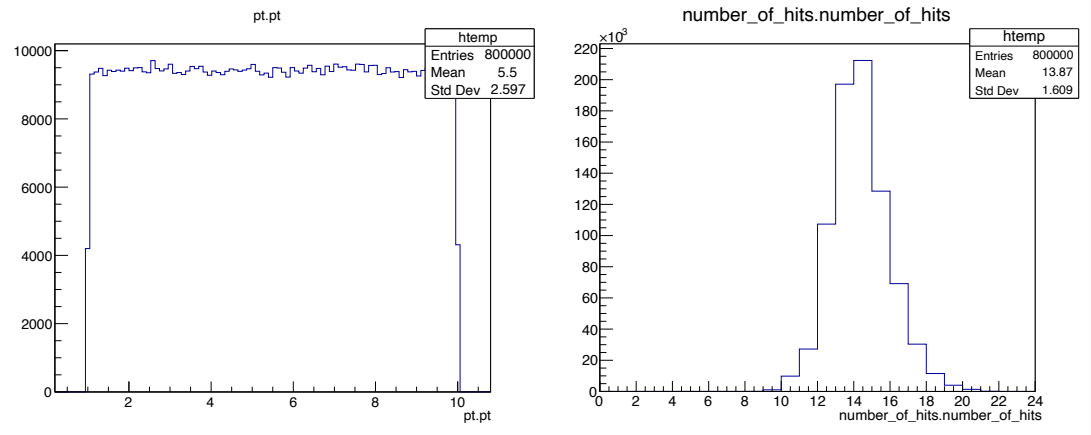
- Two simulation backends:
  - **Geant4**: Full detector simulation (with `--geant4`)
  - **FATRAS**: ACTS Fast Simulation (default)

```
addFatras(
    s,
    trackingGeometry,
    field,
    enableInteractions=True,
    outputDirRoot=outputDir,
    outputDirCsv=outputDir,
    outputDirObj=outputDir,
    rnd=rnd,
)
```

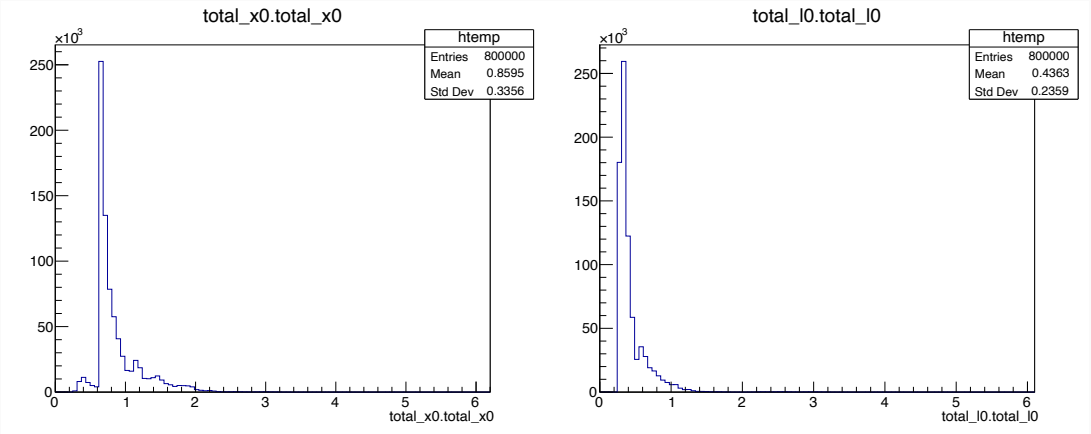
Simulation Options

- Two simulation backends:
  - **Geant4**: Full detector simulation (with `--geant4`)
  - **FATRAS**: ACTS Fast Simulation (default)
- Both produce:
  - Simulated particles (`particles_simulated.root`)
  - Hits (`hits.root`)

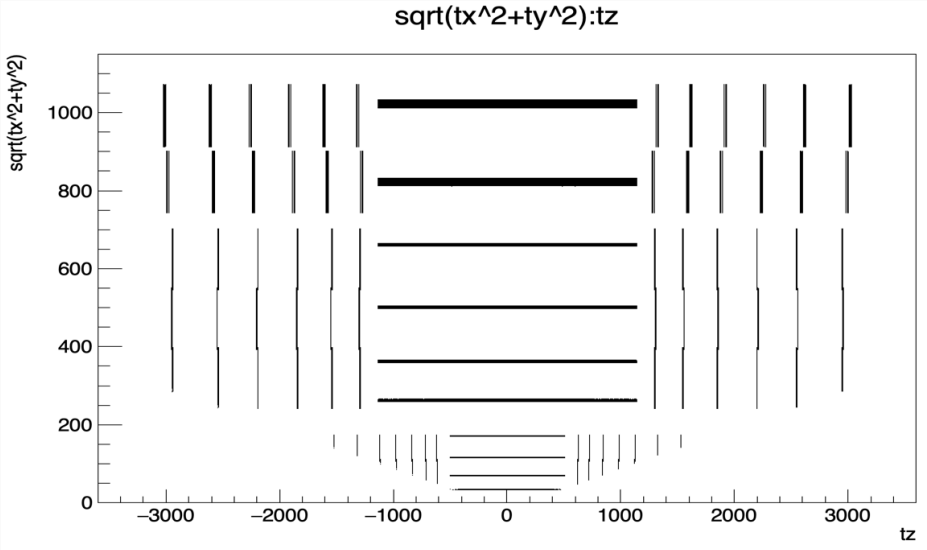
Simulated particle properties



Simulated particle properties (cont.)



Simulated particle properties (cont.)

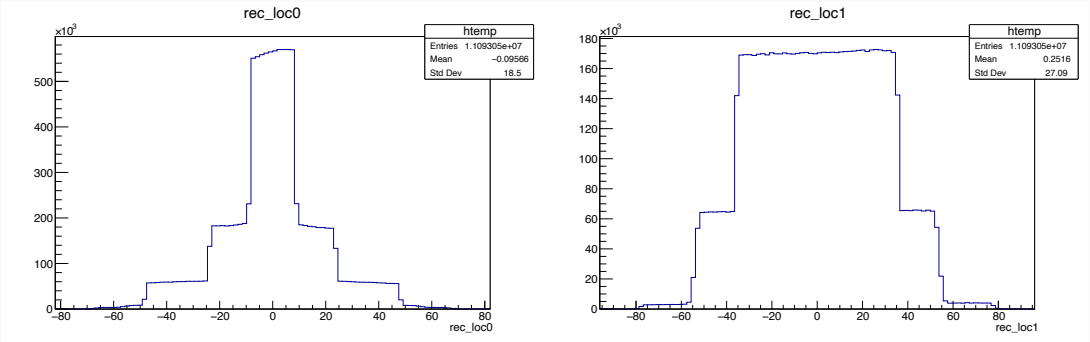


Digitization

```
addDigitization(
  s,
  trackingGeometry,
  field,
  digiConfigFile=oddDigiConfig,
  outputDirRoot=outputDir if args.output_root else None,
  outputDirCsv=outputDir if args.output_csv else None,
  rnd=rnd,
)
```

- Configures digitization process using JSON config file
  - Adds DigitizationAlgorithm from Examples framework
  - Handles both digitization and clustering if needed
  - For smearing digitization, clustering is skipped
- With configuration from before: Converts simulated hits to detector measurements via smearing

Cluster properties



- Shoulders → sensor sizes
- Cluster uncertainties and pull distributions available
  - For Gaussian smearing: expect normal pull distribution
  - Uncertainties match smearing configuration in smeared digitization mode

Particle Selection

```
addDigiParticleSelection(
    s,
    ParticleSelectorConfig(
        pt=(1.0 * u.GeV, None),
        eta=(-3.0, 3.0),
        measurements=(9, None),
        removeNeutral=True,
    ),
)
```

- Selection criteria:
  - Minimum p_T of 1.0 GeV
  - η range of ± 3.0
  - At least 9 measurements
  - Neutral particles removed

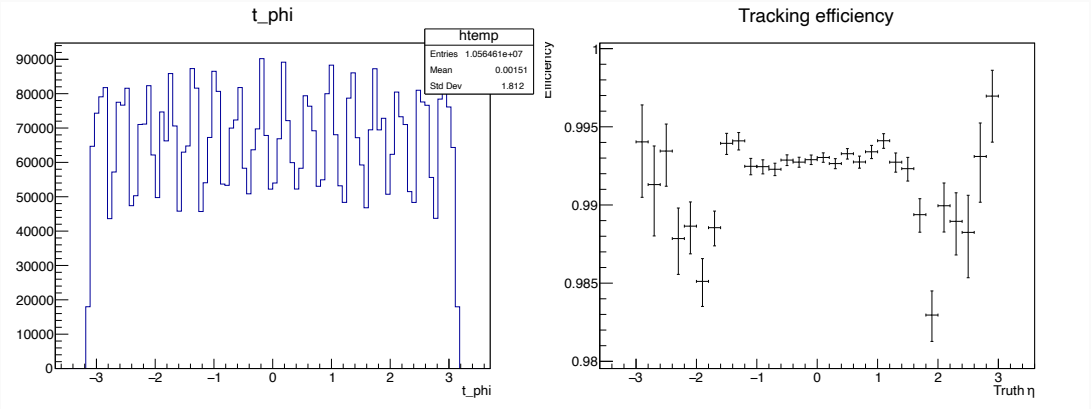
Reconstruction

Track Seeding

- First step in reconstruction pipeline
- Available seeding strategies:
  - Full triplet seeding (Default)
  - Truth smeared
  - Truth estimated
  - Orthogonal range seeding
  - Others
- Outputs:
  - `estimatedparams.root` : Track parameter information from seeds
  - `performance_seeding.root` : Seeding efficiency information

```
addSeeding(
    s,
    trackingGeometry,
    field,
    initialSigmas=[
        1 * u.mm, 1 * u.mm,
        1 * u.degree, 1 * u.degree,
        0.1 * u.e / u.GeV, 1 * u.ns,
    ],
    initialSigmaPtRel=0.1,
    initialVarInflation=[1.0] * 6,
    geoSelectionConfigFile=oddSeedingSel,
    outputDirRoot=outputDir,
    outputDirCsv=outputDir,
)
```

Seed properties



Combinatorial Kalman Filter (CKF)

- Main algorithm for track finding in ACTS
- Performance is highly sensitive to configuration
- Parameters to control:
  - Track selection criteria
  - χ^2 cut-offs
  - Volume constraints
  - Measurement handling
  - Seed handling

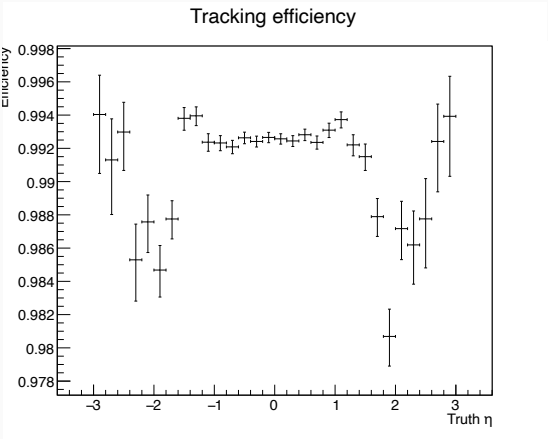
CKF Configuration

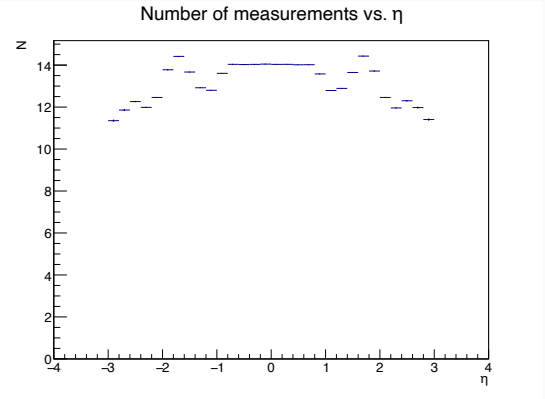
```
addCKFTracks(
    s,
    trackingGeometry,
    field,
    TrackSelectorConfig(
        pt=(1.0 * u.GeV if args.ttbar else 0.0, None), absEta=(None, 3.0),
        loc0=(-4.0 * u.mm, 4.0 * u.mm), nMeasurementsMin=7, maxHoles=2, maxOutliers=2,
    ),
    CkfConfig(
        chi2CutOffMeasurement=15.0,  # Valid measurement threshold
        chi2CutOffOutlier=25.0,      # Outlier threshold
        numMeasurementsCutOff=10,    # Max measurements to consider
        seedDeduplication=True,      # Use each seed once
        stayOnSeed=True,             # Force inclusion of seed measurements
        pixelVolumes=[16, 17, 18],   # Volume ID mapping
        stripVolumes=[23, 24, 25],
        maxPixelHoles=1, maxStripHoles=2,
        constrainToVolumes=[...],    # Restrict to specific volumes
    ),
    outputDirRoot=outputDir, outputDirCsv=outputDir, writeCovMat=True,
)
```

CKF outputs

- Track finding performance stored in `performance_finding_ckf.root`
- Track properties and quality in `performance_fitting_ckf.root`
- Ambiguity resolution needed to handle shared clusters:
  - Multiple tracks can share measurement clusters
  - Goal: Unique cluster-to-track association
- Three available strategies:
  - Machine learning based (ONNX model)
  - Score-based (quality and hit content)
  - Greedy (iterative removal by shared hit fraction)

CKF outputs (cont.)





Ambiguity Resolution

```
addAmbiguityResolution(
    s,
    AmbiguityResolutionConfig(
        maximumSharedHits=3,
        maximumIterations=1000000,
        nMeasurementsMin=7
    ),
    outputDirRoot=outputDir if args.output_root else None,
    outputDirCsv=outputDir if args.output_csv else None,
    writeCovMat=True,
)
```

- Same outputs produced for tracks after ambiguity resolution as for direct CKF outputs
- Performance metrics available in `performance_finding_ambi.root` and `performance_fitting_ambi.root`

Final Processing and Output

- Vertex fitting added as final step:
- Vertex outputs will be written to `performance_vertexing.root`

```
addVertexFitting(
    s,
    field,
    vertexFinder=VertexFinder.AMVF,
    outputDirRoot=outputDir if args.output_root else None,
)
```

- Run the full sequence:

  `s.run()`
- Output files include:
  - Truth information: `particles.root`, `vertices.root`
  - Simulation: `particles_simulated.root`, `hits.root`
  - Reconstruction: `measurements.root`, `performance_*.root`

Vertex properties